ADR-0002 : Révocation des jetons JWT par version de jeton (`token_version`)

- **Statut :** Accepté — complété le 2026-10-27
- **Date :** 2026-07-21 (décision initiale) ; 2026-10-27 (compléments)
- **Décideurs :** Driss OUAAZIZ — Software Architect

Historique des révisions

DATE	RÉVISION
2026-07-21	Décision initiale.
2026-10-27	Compléments issus de l'audit de sécurité d'octobre 2026 (document 14) : version initiale aléatoire (constat F2), rotation de session à l'activation et à la désactivation du second facteur (constat M2), déconnexion sans authentification préalable (constat M3). La décision de fond est inchangée.

Contexte et problème

L'authentification repose sur des jetons **JWT** (HS256) transportés dans un cookie `HttpOnly` (repli possible par en-tête `Authorization: Bearer`). Un JWT est par nature **auto-porteur et sans état** : une fois émis, il reste valide jusqu'à son expiration (`exp`), fixée à sept jours.

Or plusieurs situations exigent de **révoquer immédiatement** des jetons déjà émis, avant leur expiration naturelle :

- changement de mot de passe (compromission suspectée) ;
- déconnexion de toutes les sessions demandée par l'utilisateur ;
- révocation forcée par un administrateur ;
- modification d'attributs d'autorisation (rôle, permissions, groupes, statut).

Ce dernier point est critique : le rôle (`role`) est en partie lu depuis le jeton lui-même. Sans révocation, un jeton émis avant une **rétrogradation** conserverait le rôle élevé jusqu'à expiration.

Facteurs de décision

- Révocation effective et immédiate, sans attendre l'expiration.
- Conservation autant que possible du caractère sans état du JWT (pas de vérification réseau/base coûteuse à chaque requête).
- Migration transparente des jetons et comptes existants.
- Faible complexité opérationnelle.

Options envisagées

1. **Liste de révocation (denylist) de jetons** — Nécessite de stocker et consulter chaque jeton révoqué à chaque requête ; réintroduit un état par jeton et une croissance non bornée à purger.

2. **Sessions serveur (jetons opaques)** — Abandonne le modèle JWT, impose une consultation de session à chaque requête ; plus lourd à mettre en œuvre.
3. **TTL très court + rafraîchissement** — Réduit la fenêtre d'abus mais ne révoque pas *immédiatement* et complexifie le cycle de vie (refresh tokens).
4. **Compteur de version par utilisateur (`token_version`)** *(retenu)* — Une seule valeur par utilisateur, comparée à un claim embarqué dans le jeton.

Décision

Introduire un compteur **`token_version`** (`u64`) porté par chaque utilisateur (`StoredUser`) et embarqué dans le JWT sous le claim **`tv`** au moment de l'émission.

- À chaque requête authentifiée, le middleware `require_auth` (et `me()`) rejette le jeton si `claims.tv != user.token_version` .
- **Incrémenter `token_version` invalide instantanément tous les jetons en circulation** pour cet utilisateur.
- Déclencheurs d'incrément : changement de mot de passe (`set_password`), déconnexion globale (`POST /api/auth/logout-all`), révocation administrateur (`POST /api/admin/users/:u/revoke-sessions`), toute modification d'attribut d'autorisation via `apply_update` (rôle, permissions, groupes, statut), ainsi que l'activation (`POST /api/auth/2fa/confirm`) et la désactivation (`POST /api/auth/2fa/disable`) du second facteur.
- **Valeur initiale aléatoire** (complément 2026-10) : à la création d'un compte, `token_version` reçoit une valeur `u64` tirée aléatoirement, et non `0` . Un JWT émis pour un compte supprimé ne redevient donc pas valide si un compte est recréé ultérieurement sous le même nom (constat F2). L'incrément s'effectue en arithmétique modulaire (`wrapping_add`).
- **Rotation de session sur changement du second facteur** (complément 2026-10) : la confirmation et la désactivation de la 2FA incrémentent `token_version` puis émettent aussitôt un nouveau cookie de session pour l'appareil courant. Les autres sessions — par exemple un cookie dérobé — sont révoquées sans déconnecter l'utilisateur qui agit (constat M2). Désactiver la 2FA ou régénérer les codes de secours exige en outre le mot de passe **et** un code valide (TOTP ou code de secours).
- **Déconnexion simple** (`POST /api/auth/logout` , complément 2026-10) : le point d'entrée ne requiert plus de session valide et efface toujours les deux cookies (`fileflow_token` , `fileflow_2fa_pending`), y compris avec un jeton déjà révoqué ou expiré (auparavant, erreur 500 et cookie conservé — constat M3). Elle n'incrémente pas `token_version` : seule la déconnexion globale révoque côté serveur les jetons en circulation.
- Le claim `tv` et le champ `token_version` sont annotés `#[serde(default)]` : les jetons et comptes préexistants sont traités comme version `0` , garantissant une **montée de version sans rupture**.

Le contrôle reste une simple comparaison d'entiers en mémoire : le modèle demeure essentiellement sans état, sans appel réseau supplémentaire.

Conséquences

Positives

- Révocation immédiate et globale par utilisateur, à coût constant.
- Les changements d'autorisation (rétrogradation, retrait de permissions) prennent effet sans délai, éliminant la fenêtre du jeton périmé.
- Migration transparente (`serde(default)`).

- Pas de réactivation accidentelle d'anciens jetons lors de la recréation d'un compte homonyme (valeur initiale aléatoire).
- Aucune liste de révocation à stocker ni à purger.

Négatives

- Granularité **par utilisateur** : impossible de révoquer un seul appareil de façon sélective sans invalider les autres (la déconnexion globale invalide tout).
- La déconnexion simple supprime le cookie du navigateur mais ne révoque pas le jeton côté serveur : une copie dérobée reste valable jusqu'à son expiration, sauf déconnexion globale.

Risques résiduels

- Un `token_version` non incrémenté lors d'un futur nouveau chemin de modification d'autorisation laisserait subsister un jeton privilégié. **Mesure : couverture de tests dédiée** (`apply_update` incrémente bien la version sur changements de sécurité, pas sur changements cosmétiques).

Références

- Code : `backend/src/{auth,users,lib,admin,two_factor}.rs`, modèle `Claims` / `StoredUser`.
- Document 14 — rapport d'audit de sécurité d'octobre 2026 (constats F2, M2, M3).
- ADR-0003 (choix HS256) et ADR-0006 (limitation des tentatives) — sécurité d'authentification associée.